Table 13.4 Integration checklist for adopting a policy engine

Step	What to establish	Why it matters
1. Map boundaries	Identify system, application, and tenant scopes.	Clarifies which policies run where and who owns them
2. Define the identity model	Standardize how principals, tenants, and resources are represented.	Ensures every PDP can reason over the same entities
3. Externalize decisions	Move allow/deny logic from code to policy evaluation	Enables consistency and auditability
4. Place enforcement points	Insert PEPs at gateways, services, and (optionally) the UI.	Keeps policy evaluation close to data while preserving performance
5. Establish the schema	Create a shared Cedar schema for all policy layers.	Guarantees type safety and cross-layer compatibility
6. Support partial evaluation	Precompute scopes for fast UI search and discovery.	Improves responsiveness without compromising correctness
7. Implement logging and versioning	Record every decision and track policy versions.	Enables rollback, audit, and long-term governance

A mature integration doesn't end with code changes; it depends on operational discipline. Policies need ownership, testing, and monitoring just as any other production system does.

13.7.3 Looking ahead

C³'s development from an internal tool to a multi-tenant cloud platform demonstrates what happens when authorization becomes a first-class part of system design. By separating policy from code, ACME gained greater flexibility, better auditability, and a structure that can adapt as new tenants, identities, and regulations emerge.

The next challenge isn't creating more policies; it's effectively managing them. Policies now exist across multiple PDPs, change hands among different owners, and rely on signals that fluctuate constantly. In the next chapter, we'll see how ACME formalizes that management: defining who controls each policy layer, how signals are verified and standardized, and how policy lifecycle management turns externalized authorization into a daily operational practice.